第一部分：用例规约与事件流分析：WETest 小程序修改用户名或密码

项目	内容
用例编号	UC-05
用例名称	修改用户名或密码
系统	WETest 微信小程序端 ——「我的 → 账号与安全」
文档版本	v1.0
编写日期	2026-10-30

1. 用例概述

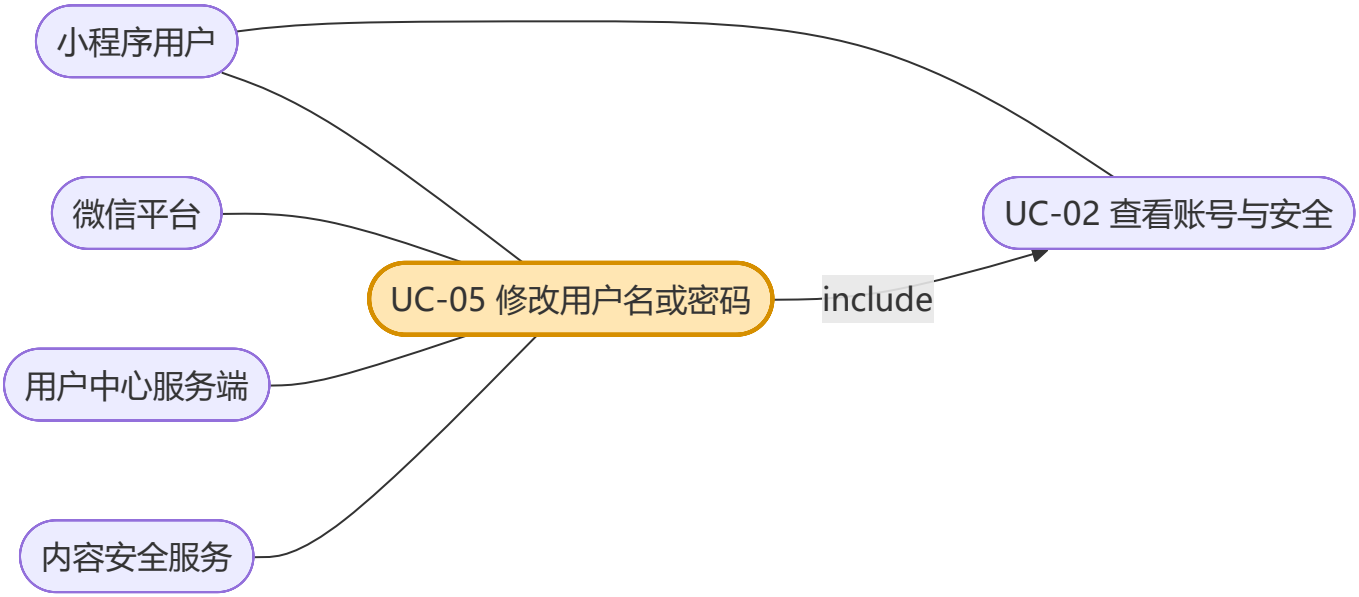
字段	内容
主要参与者	小程序用户（WeTest 开发者 / 测试工程师）
次要参与者	微信平台（提供登录态与授权）、用户中心服务端（唯一性校验、持久化、并发控制）、内容安全服务（用户名敏感词比对）
触发条件	用户在「我的」页点击「账号与安全」，进入后选择「修改用户名」或「修改密码」
前置条件	1. 用户已通过微信授权登录，小程序本地持有有效 `token` 2. 用户账号已存在，且已绑定手机号或邮箱（用于改密时的二次身份验证） 3. 网络环境可用（弱网场景由异常流覆盖）
后置条件	**成功路径**：目标字段更新为新值并持久化；`version + 1`；生成一条变更日志；若修改的是密码，旧登录态按策略失效并下发新 `token` **失败路径**：数据库中用户名 / 密码**保持原值不变**，页面保留用户本次已输入内容以供重试
成功判定	服务端返回成功且回显数据与用户提交一致，页面展示已更新，提示「修改成功」
扩展点	步骤 5（选择修改项）、步骤 7（输入新值）、步骤 9（前端校验）、步骤 11（登录态）、步骤 12（业务校验）、步骤 13（身份验证）、步骤 14（写入）
关联用例	UC-02 查看账号与安全（include）、UC-08 修改记录查询（extend）

2. 字段与校验规则

字段	可修改	必填	约束	服务端约束
用户名	✅	是	2–20 位，字母 / 数字 / 下划线，首位非数字，自动去除首尾空格	**全局唯一**，敏感词库，与原用户名不可相同
登录密码	✅	是	6–20 位，至少包含字母与数字，两次输入一致	强度策略（禁止弱密码 / 常见密码），与历史密码不可相同，**加盐哈希存储，不落明文**
绑定手机号 / 邮箱	❌	—	只读，仅用于身份验证	—
OpenID / UnionID	❌	—	系统标识，只读	—

---

3. 用例图



---

4. 主事件流（基本流）

步骤	参与者	动作 / 响应	扩展点
1	用户	在小程序底部点击「我的」Tab，进入个人中心	
2	系统	校验本地 `token` 有效性；有效则请求并展示用户当前资料与「账号与安全」入口	→ **E2**
3	用户	点击「账号与安全」	
4	系统	展示当前用户名（脱敏显示）、绑定手机号 / 邮箱，并提供「修改用户名」「修改密码」「修改记录」三个入口	
5	用户	选择修改项：点击「修改用户名」或「修改密码」	→ **A2**, **A7**
6	系统	进入对应修改页：加载可编辑字段并回填当前值；只读字段置灰；读取本地草稿缓存（若有）并提示「检测到未保存的草稿，是否恢复」	→ **A6**
7	用户	输入新值（用户名：新用户名；密码：新密码 + 确认密码）	→ **A3**, **A4**, **A5**
8	用户	点击「确定」	
9	系统	前端预校验：必填、长度、字符集、两次密码一致、密码强度；不通过则字段级红字提示并聚焦首个错误字段	→ **E1**
10	系统	组装请求体（`token` + 目标字段 + 新值 + `version`，改密时附验证凭据），发起 `POST /api/account/credential`	
11	服务端	校验 `token` 签名与有效期	→ **E2**

步骤	参与者	动作 / 响应	扩展点
12	服务端	业务校验：用户名唯一性与敏感词、与原值是否相同、密码强度策略、密码与历史密码是否重复；并比对 `version` 是否匹配	→ **E3**, **E9**
13	服务端	若为修改密码：执行二次身份验证（短信验证码或原密码），验证不通过则拒绝	→ **A5**, **E7**, **E8**
14	服务端	开启事务写入数据库并记录变更日志 `account_change_log`，`version + 1`；若改密则使其他设备会话失效并下发新 `token`	→ **E4**
15	系统	接收成功响应，刷新本地用户缓存与全局状态，退出修改页，返回上一页并展示更新后的信息，Toast「修改成功」	→ **A6**
16	—	**用例成功结束**	

---

5. 备选事件流

A1 用户放弃修改（在步骤 6 扩展）

- **A1.1** 用户在修改页点击返回
- **A1.2** 系统弹出二次确认「当前修改尚未保存，是否放弃？」
- **A1.3** 用户点击「放弃」→ 丢弃本次修改与草稿，返回上一页，展示**原值** → **用例结束（成功，资料未变更）**
- **A1.4** 用户点击「继续编辑」→ 关闭弹窗 → **回归步骤 7**

A2 用户名与密码同时修改（在步骤 5 扩展）

- **A2.1** 用户在同一提交中同时填写新用户名与新密码
- **A2.2** 系统按**原子更新**处理：两项在同一事务内生效，任一项校验失败则两项均不生效
- **A2.3** **回归步骤 9**

A3 修改密码时选择其他验证方式（在步骤 7 扩展）

- **A3.1** 系统提供「短信验证码」与「原密码」两种验证方式供用户切换
- **A3.2** 用户切换验证方式 → 界面切换对应输入控件
- **A3.3** **回归步骤 7**

A4 从微信同步用户名（在步骤 7 扩展）

- **A4.1** 用户点击「同步微信昵称」
- **A4.2** 系统读取微信昵称并填入用户名输入框，同时标记需重新校验格式
- **A4.3 回归步骤 7**

A5 二次身份验证通过（在步骤 13 扩展）

- **A5.1** 用户选择「短信验证码」→ 系统发送验证码（60 秒倒计时内不可重复发送）→ 用户输入
- **A5.2** 或用户选择「原密码」→ 用户输入原登录密码
- **A5.3** 验证通过 → 服务端继续执行写入 → **回归步骤 14**

A6 修改密码成功后的会话处理（在步骤 15 扩展）

- **A6.1** 系统提示「密码已修改，其他设备需重新登录」
- **A6.2** 本机使用新 `token` 保持登录，其他设备会话标记为失效
- **A6.3 回归步骤 16**

A7 修改完成后查看变更记录（在步骤 5 扩展）

- **A7.1** 用户点击「修改记录」
- **A7.2** 系统展示历史变更条目（时间、字段、变更前后值）
- **A7.3 用例结束**

---

6. 异常事件流

E1 前端校验失败（扩展点：步骤 9）

- **E1.1** 校验不通过：用户名为空 / 少于 2 位 / 超过 20 位 / 含非法字符 / 首位为数字、两次密码不一致、密码强度不足
- **E1.2** 系统在对应字段下方展示具体错误文案，字段边框标红，页面自动滚动并聚焦第一个错误字段
- **E1.3** 用户修正后重新校验 → **回归步骤 9**

E2 登录态失效 / token 过期（扩展点：步骤 2、步骤 11）

- **E2.1** 检测到 `token` 过期或无效
- **E2.2** 前端清除本地登录态与用户缓存，跳转登录引导页；服务端返回 401 并记录鉴权日志
- **E2.3** 用户重新完成微信授权登录 → 返回「账号与安全」页，**本次已输入内容丢失**
- **E2.4 用例结束（失败，资料未变更）**

E3 服务端业务校验失败（扩展点：步骤 12）

- **E3.1** 业务规则不通过：用户名已被占用、用户名含敏感词、用户名与原值相同、密码不符合强度策略、密码与历史密码重复
- **E3.2** 服务端返回结构化错误码与字段级信息（如 `{ field: 'username', code: 'DUPLICATE', msg: '该用户名已被使用' }`）

- **E3.3** 前端定位到出错字段并高亮提示，其余已填内容**全部保留**
- **E3.4 回归步骤 7**

E4 数据库写入失败（扩展点：步骤 14）

- **E4.1** 写入异常（连接超时、死锁、唯一索引冲突）
- **E4.2** 服务端回滚事务，保证用户名 / 密码仍为原值（**数据一致性保证**）
- **E4.3** 返回失败码，前端按 E6 处理 → **回归步骤 10**（重试）或用户放弃 → **用例结束（失败）**

E5 网络异常 / 请求超时（扩展点：步骤 10）

- **E5.1** 请求超时（≥10s）或连接失败（断网、弱网、DNS 失败）
- **E5.2** 前端提示「网络异常，保存失败，请检查网络后重试」，**草稿写入本地缓存**
- **E5.3** 用户点击「重试」→ 重新发起步骤 10（最多 3 次，超过则要求重新登录）
- **E5.4** 用户放弃 → 保留草稿 → **用例结束（失败）**

E6 服务端内部错误（扩展点：步骤 14）

- **E6.1** 服务端返回 5xx 或抛出未捕获异常
- **E6.2** 服务端记录错误日志与链路追踪 ID，并回滚事务
- **E6.3** 前端提示「服务开小差了，请稍后重试」，展示可复制的错误码便于排查
- **E6.4 回归步骤 10**

E7 验证码发送失败 / 频率限制（扩展点：步骤 A5.1）

- **E7.1** 短信通道失败或触发频控（60 秒内重复请求、每日上限）
- **E7.2** 提示「验证码发送过于频繁，请 60 秒后重试」/「今日发送次数已达上限」
- **E7.3 回归 A5.1**（倒计时结束后可重发）

E8 原密码验证失败（扩展点：步骤 A5.2）

- **E8.1** 用户输入的原密码与服务端记录不匹配
- **E8.2** 提示「原密码不正确」，输入框清空并聚焦，允许重新输入（连续失败 5 次临时锁定改密入口 10 分钟）
- **E8.3 回归 A5.2**

E9 并发修改冲突（扩展点：步骤 12）

- **E9.1** 提交的 `version` 与服务端当前 `version` 不一致（同一账号在其他设备同时修改）
- **E9.2** 系统提示「账号资料已在其他设备更新」
- **E9.3** 用户选择「查看最新」→ 放弃本地修改，重新加载服务端数据 → **回归步骤 6**
- **E9.4** 用户选择「覆盖保存」→ 以本地为准强制提交（服务端 `force` 标记）→ **回归步骤 13**

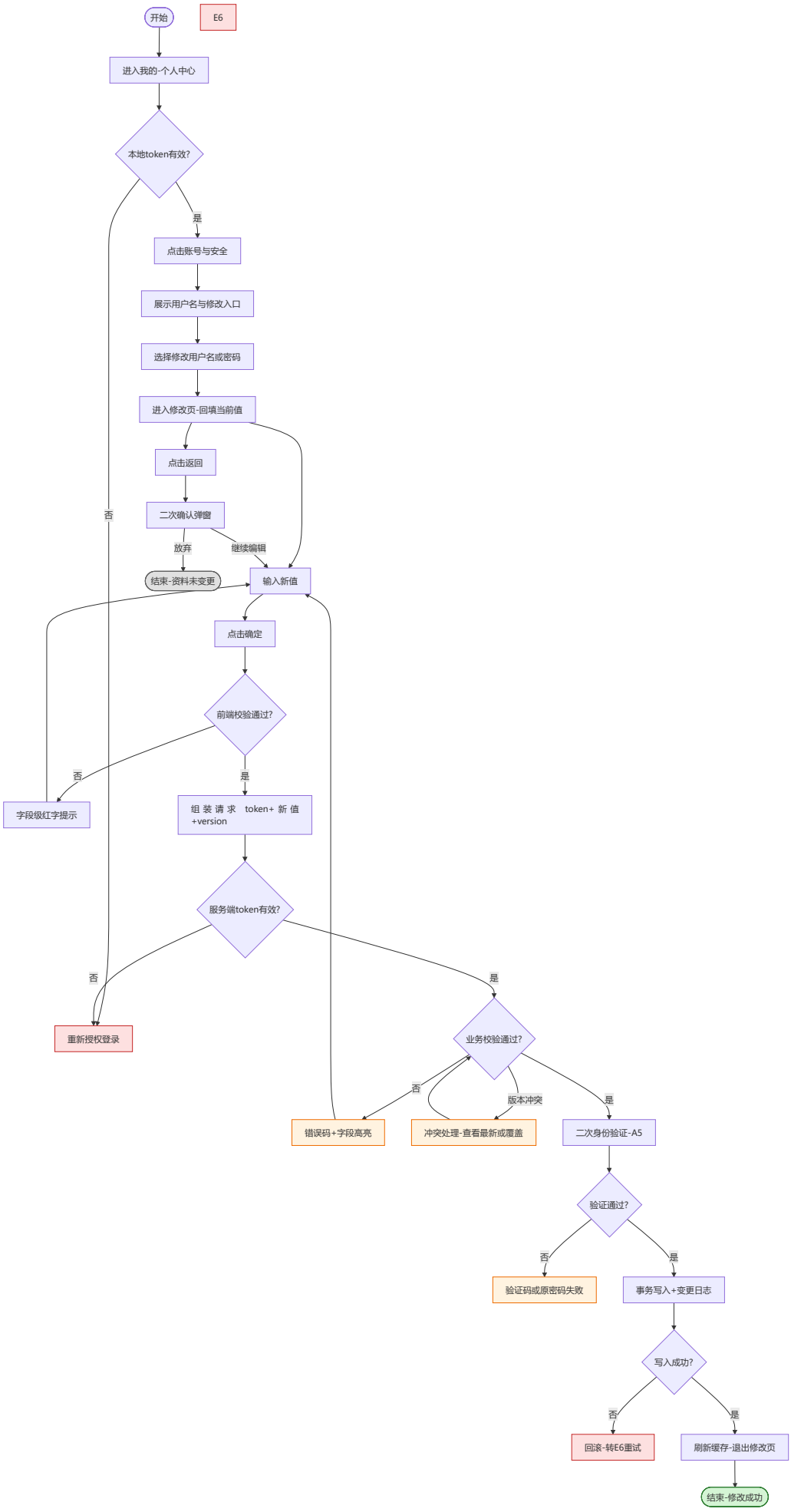
E10 改密后会话失效处理异常（扩展点：步骤 14）

- **E10.1** 密码写入成功但会话失效下发失败，导致旧 `token` 残留
- **E10.2** 服务端标记该用户全部会话需重新认证，下次请求强制跳转登录页
- **E10.3** 前端提示「密码已修改，请使用新密码重新登录」

- **E10.4 用例结束（失败，密码已变更）**

---

7. 事件流图


开始
E6
进入我的-个人中心
本地token有效?
是
否
点击账号与安全
展示用户名与修改入口
选择修改用户名或密码
进入修改页-回填当前值
点击返回
二次确认弹窗
放弃
继续编辑
结束-资料未变更
输入新值
点击确定
前端校验通过?
否
是
字段级红字提示
组 装 请 求 token+ 新 值 +version
服务端token有效?
否
是
重新授权登录
业务校验通过?
否
版本冲突
是
错误码+字段高亮
冲突处理-查看最新或覆盖
二次身份验证-A5
验证通过?
否
是
验证码或原密码失败
事务写入+变更日志
写入成功?
否
是
回滚-转E6重试
刷新缓存-退出修改页
结束-修改成功

8. 事件流与测试用例对应表

事件流	测试点	测试设计方法	优先级
主流程 步骤 1–16	合法用户名 / 合法密码提交，资料正确落库并回显	等价类（有效等价类）	P0
步骤 7 / E1	用户名 1/2/20/21 位、首字符为数字、含非法字符与空格	边界值分析 + 错误推测	P0
步骤 7 / E1	密码 5/6/20/21 位、纯数字、纯字母、两次输入不一致	边界值分析	P0
A1.3 / A1.4	返回二次确认两个分支；放弃后原值不变、草稿清除	场景法（分支覆盖）	P1
A2.2	用户名与密码同提，其中一项非法 → 两项均不生效	场景法 + 事务原子性校验	P1
A3 / A5	两种二次验证方式切换，验证码正确 / 错误 / 超时	等价类 + 场景法	P1
A4.2	微信昵称含空格或表情，同步后仍需通过格式校验	错误推测	P2
A6.1	改密后其他设备会话失效、本机保持登录	场景法	P1
A7.2	修改记录展示的字段与前后值正确	等价类	P2
E2	token 过期后提交，提示重登录且不写入脏数据	错误推测	P0
E3	用户名重复、含敏感词、与原值相同	等价类 + 错误推测	P0
E4	写入中途失败，事务回滚后数据为原值	故障注入	P1
E5	断网、弱网（限速 / 丢包 30%）、请求超时	弱网测试	P0
E6	服务端 5xx 注入，返回友好错误且带可复制错误码	故障注入	P1
E7	验证码 60 秒内重复发送、每日上限	边界值分析	P1

事件流	测试点	测试设计方法	优先级
E8	原密码错误、连续 5 次失败锁定	边界值分析	P1
E9	手机与 PC 双端同时改同字段	并发 / 竞态测试	P1
E10	改密后会话下发失败，强制重新登录	故障注入	P2
全流程	新密码不出现在任何日志、接口返回与前端明文展示中	安全测试	P0

覆盖度自查

- [ ] 每条备选流 A1–A7 至少 1 条用例
- [ ] 每条异常流 E1–E10 至少 1 条用例
- [ ] 用户名、密码各自的上下边界值（界内 ±1）各至少 1 条用例
- [ ] 每个步骤的系统响应均有可观测断言（不只是「不报错」）
- [ ] 失败路径均验证「数据库原值不变」
- [ ] 改密类场景均验证「旧密码失效、新密码可登录、旧会话失效」
- [ ] 并发、弱网、token 失效三类触发条件已覆盖

第二部分：图书馆管理系统 UML 图形化建模

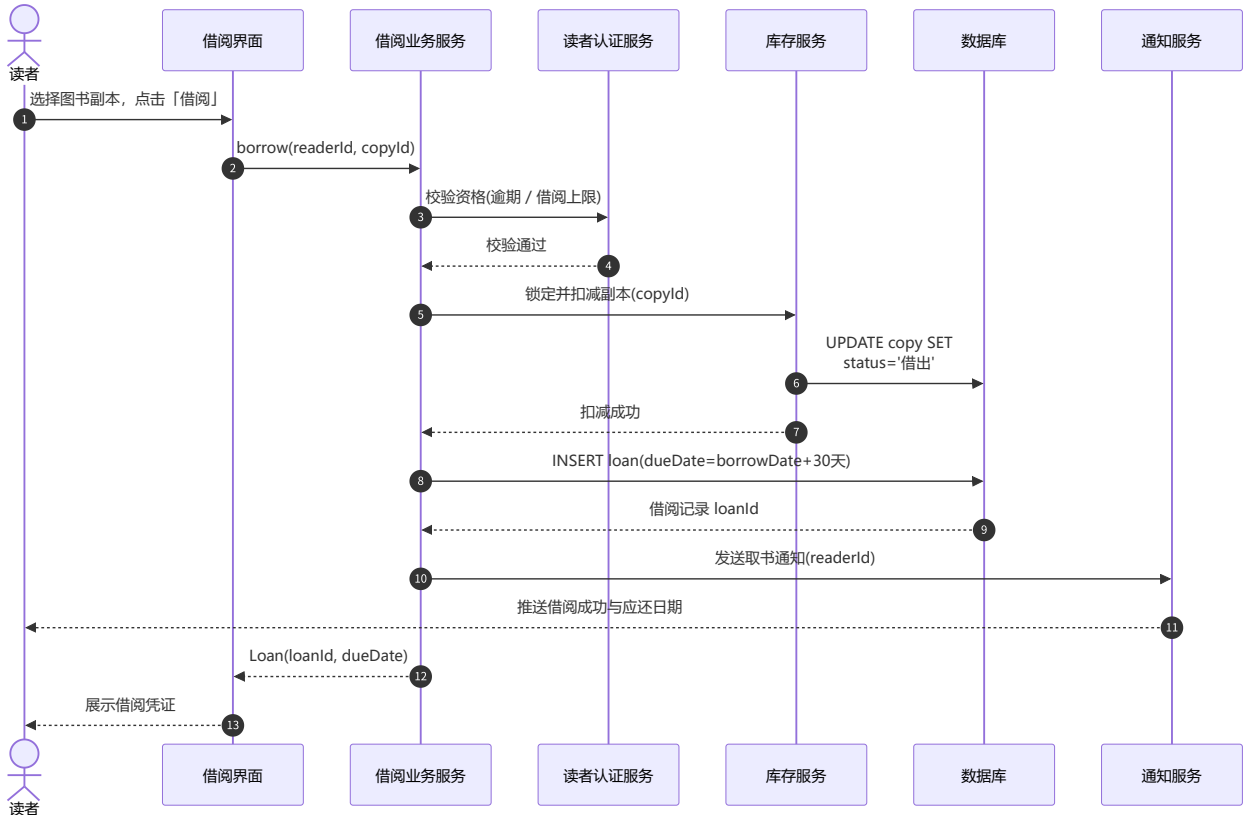
> 示例系统为「图书馆管理系统」，建模内容为 UML 2.x 结构图与行为图，共 9 张。

项目	内容
示例系统	图书馆管理系统（Library Management System, LMS）
建模范围	读者借阅全流程：检索 → 借书 → 续借 / 逾期 → 还书 → 罚款结算
图的数量	9 张（时序图、用例图、类图、对象图、活动图、组件图、部署图、状态图、定时图）
建模规范	UML 2.x（组件图按作业要求采用 UML 1.x 遗留语法）
图纸索引	见文末「9.10 图纸索引表」

9.1 时序图（Sequence Diagram）

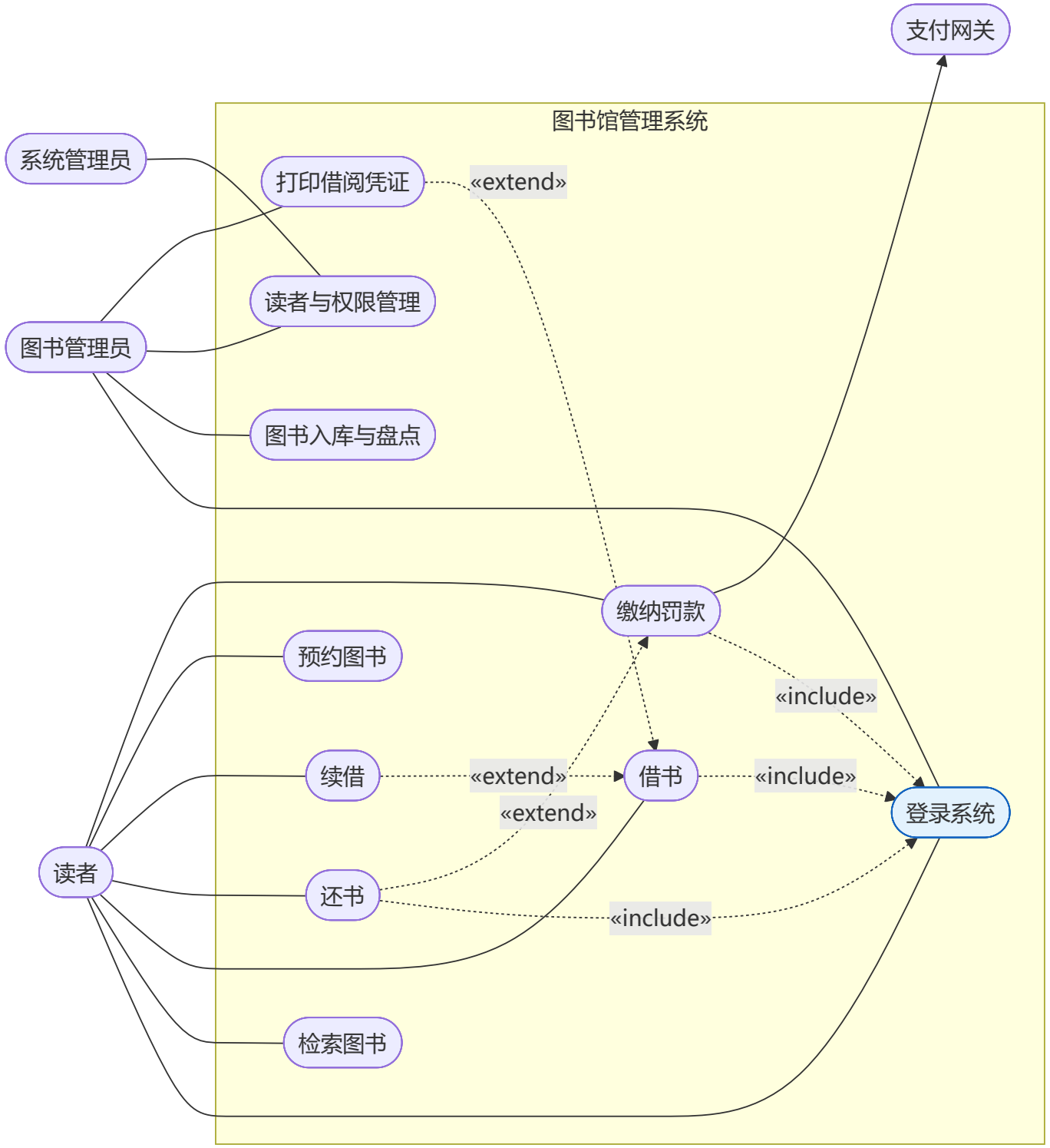
建模要点

元素	角色	说明
`R` 读者	参与者	发起借阅、接收通知
`UI` 借阅界面	边界对象	收集输入、展示借阅凭证
`LS` 借阅业务服务	控制对象	借阅用例的编排中心
`AS` 读者认证服务	协作对象	校验逾期、借阅上限
`IS` 库存服务	协作对象	锁定并扣减副本
`DB` 数据库	数据存储	借阅记录与副本状态持久化
`NS` 通知服务	协作对象	推送取书通知



读图要点：实线箭头为同步消息，虚线箭头为返回消息；`autonumber` 给出消息序号，可直接对应第 5 章事件流的步骤编号；服务之间的调用顺序体现「先校验资格、再占用库存、最后落库」的一致性策略。

9.2 用例图（Use Case Diagram）



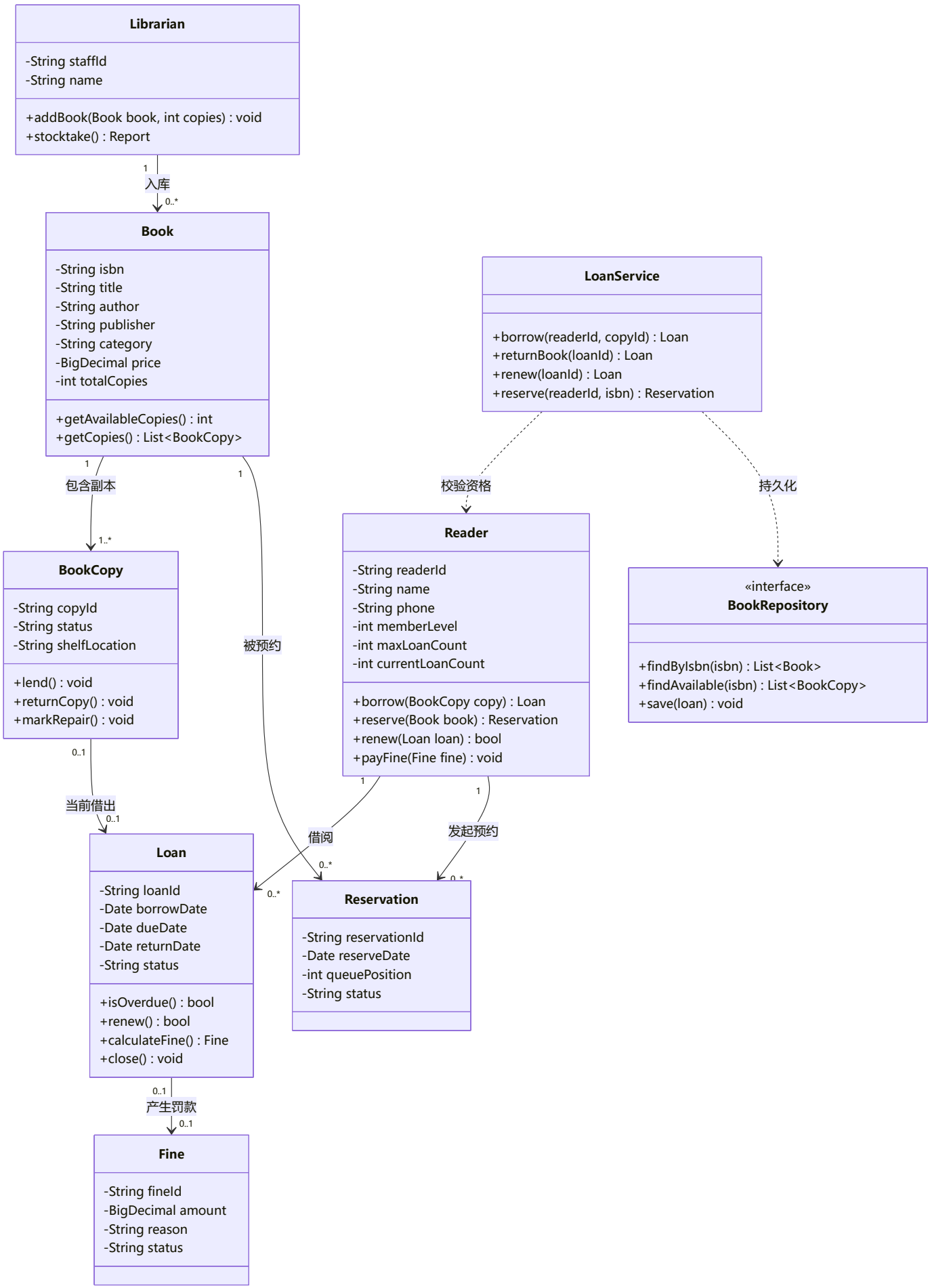
读图要点

- **关联（Association）**：读者是借书、还书、续借、预约、缴罚款的主要参与者；图书管理员负责入库、盘点与读者管理。
- **包含（«include»）**：借书、还书、缴罚款**必须**先登录，为强制前置行为，用虚线箭头指向被包含用例。
- **扩展（«extend»）**：缴罚款只在还书时存在逾期才触发；续借与打印凭证是借书的可选附加行为，箭头由扩展用例指向基础用例。

- **外部系统**：支付网关作为次要参与者，只与「缴纳罚款」用例交互。

> 渲染说明：
>
> 1. Mermaid 无原生椭圆节点，本图用圆角矩形（stadium）表示用例，与 UML 用例的椭圆语义等价。
> 2. UML 构造型（stereotype）写作 `«include»`「书名号」而非 `<<include>>`「尖括号」。尖括号在 Mermaid 中会被当作 HTML 标签处理——写在 `|` 管道标签里会被清洗器吞掉导致标签空白，写在 `:` 冒号标签里则直接触发语法解析错误。
> 3. `include` / `extend` 使用 `-.->`（虚线箭头）而非 `..>`。`..>` 是类图的依赖语法，在流程图（graph / flowchart）中不受支持，会导致整图解析失败。
> 4. 边标签统一使用 `|文本|` 管道语法；`A --> B : 文本` 形式在冒号后带空格时同样会解析报错。

9.3 类图（Class Diagram）

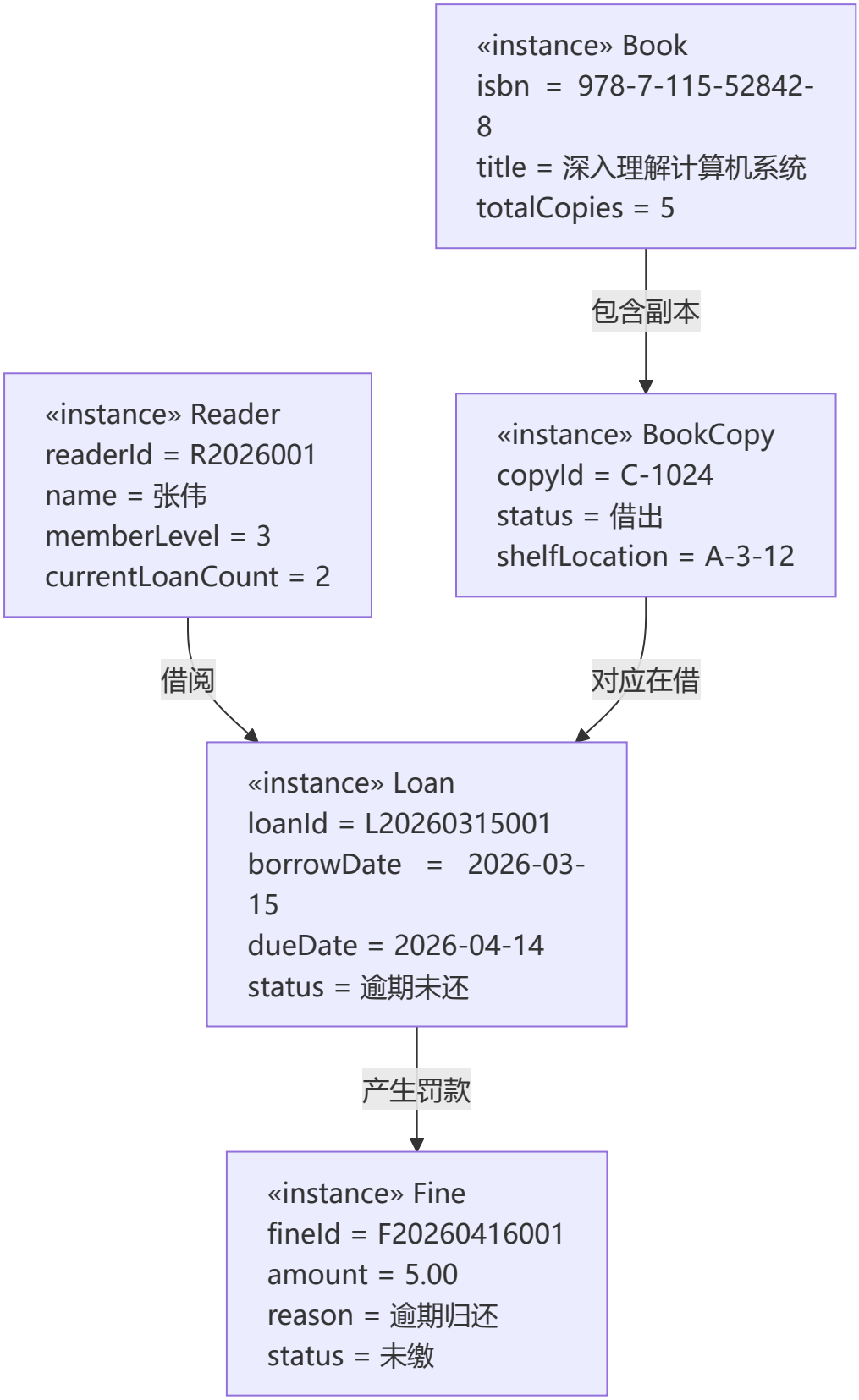

Librarian
-String staffId
-String name
+addBook(Book book, int copies) : void
+stocktake() : Report
1
入库
0..*
Book
-String isbn
-String title
-String author
-String publisher
-String category
-BigDecimal price
-int totalCopies
+getAvailableCopies() : int
+getCopies() : List<BookCopy>
1
包含副本
1..*
1
被预约
BookCopy
-String copyId
-String status
-String shelfLocation
+lend() : void
+returnCopy() : void
+markRepair() : void
0..1
当前借出
0..1
LoanService
+borrow(readerId, copyId) : Loan
+returnBook(loanId) : Loan
+renew(loanId) : Loan
+reserve(readerId, isbn) : Reservation
校验资格
持久化
Reader
-String readerId
-String name
-String phone
-int memberLevel
-int maxLoanCount
-int currentLoanCount
+borrow(BookCopy copy) : Loan
+reserve(Book book) : Reservation
+renew(Loan loan) : bool
+payFine(Fine fine) : void
«interface»
BookRepository
+findByIsbn(isbn) : List<Book>
+findAvailable(isbn) : List<BookCopy>
+save(loan) : void
1
借阅
0..*
1
发起预约
0..*
Loan
-String loanId
-Date borrowDate
-Date dueDate
-Date returnDate
-String status
+isOverdue() : bool
+renew() : bool
+calculateFine() : Fine
+close() : void
0..1
产生罚款
0..1
Reservation
-String reservationId
-Date reserveDate
-int queuePosition
-String status
Fine
-String fineId
-BigDecimal amount
-String reason
-String status

读图要点

分类	类	建模理由
实体类	`Reader` `Book` `BookCopy` `Loan` `Reservation` `Fine` `Librarian`	对应领域模型，图书与副本分离以支持「一书多册、逐册流通」
边界/服务类	`LoanService`	控制对象，编排借阅用例，与时序图中的 `LS` 对应
持久化接口	`BookRepository` `<<interface>>`	依赖倒置，服务依赖接口而非具体存储
关键多重性	`Reader "1" --> "0..*" Loan`	一个读者可同时持有多条借阅记录
关键多重性	`BookCopy "0..1" --> "0..1" Loan`	一个副本同一时刻至多对应一条进行中的借阅

9.4 对象图（Object Diagram）

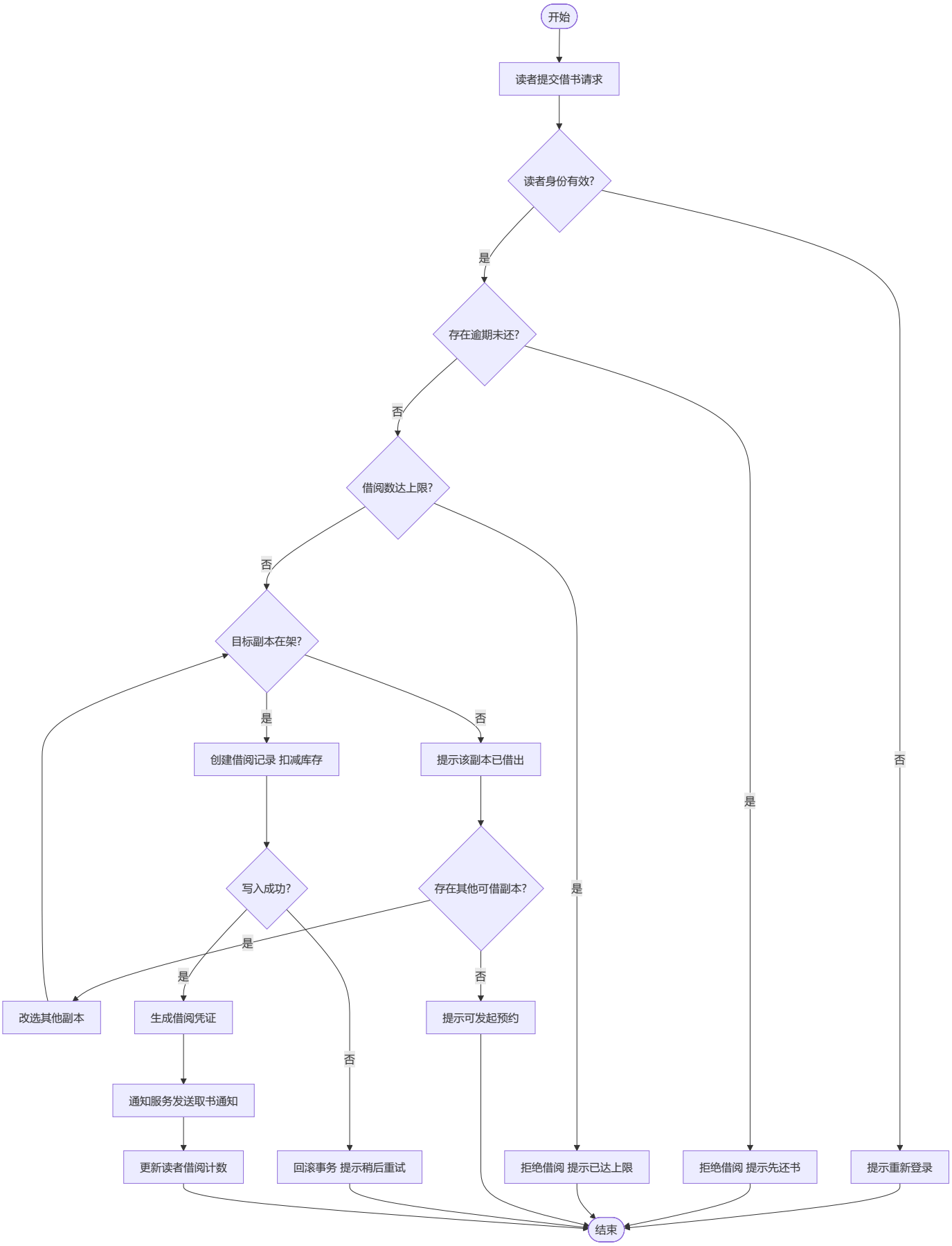
对象图是类图在**某一时刻的具体实例快照**，下图展示 t_0 时刻「张伟借出《深入理解计算机系统》第 3 册、已逾期产生罚款」的实际对象状态。



读图要点

- 对象名以下划线标注（`R2026001` 等），属性填入**具体值**而非类型，这是对象图与类图最直观的区别。
- 链接上标注的是**实例级关联**，可直接读出当前事实：张伟持有 L20260315001，该借阅对应副本 C-1024，该借阅已产生 5.00 元逾期罚款。
- 该快照与 9.9 定时图的 t_1 时点（记录转「已逾期」）完全对应，两图可交叉验证。

9.5 活动图（Activity Diagram）

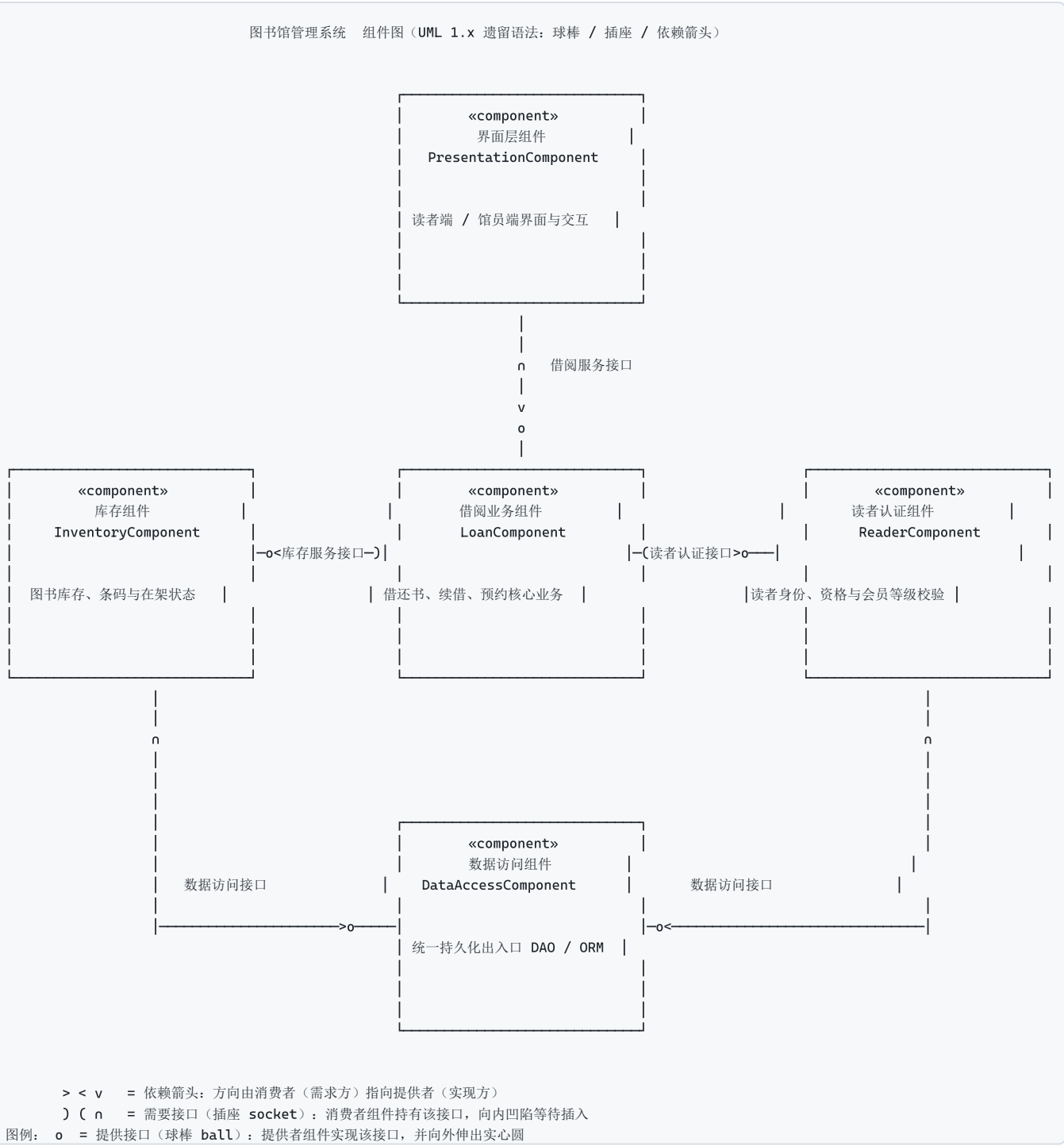


读图要点

- **判定节点（菱形）** 共 6 个，构成借书的准入校验链：身份 → 逾期 → 上限 → 库存，且库存判定带「改选副本」的**循环回边**。
- **分支终点统一汇入「结束」**，与第一部分事件流「每个分支都有明确终止点」的写法一致。
- 失败分支（`B1` `C1` `D1` `G1`）统一为「拒绝 / 回滚 + 提示」，其中 `G1` 明确标注**事务回滚**，保证失败时库存与借阅计数不变。

9.6 组件图（Legacy syntax Component diagram）

本图按作业要求采用 **UML 1.x 遗留语法**：以「球棒（ball）」表示提供接口、以「插座（socket）」表示需要接口，用带箭头的连线表达组件间的依赖方向。

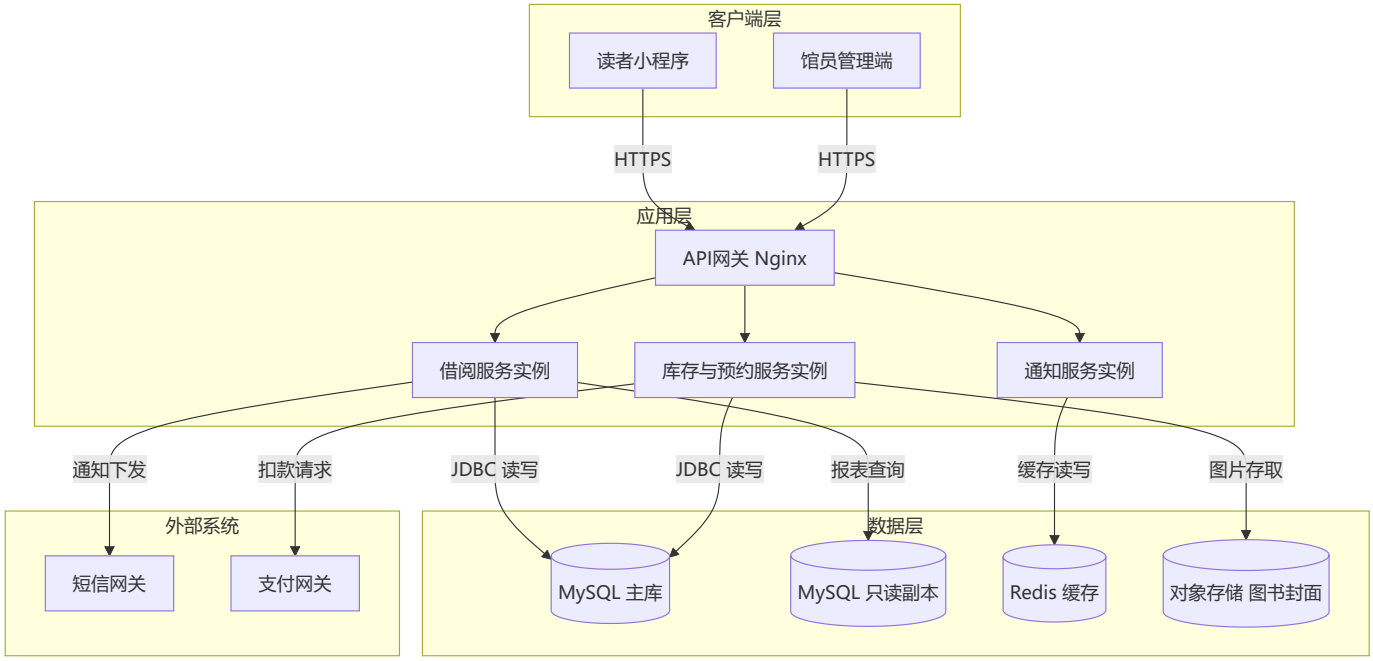


读图要点

记号	名称	语义
`o`	球棒 ball	提供接口，由提供者组件实现并向外伸出实心圆
`)` `(` `∩`	插座 socket	需要接口，由消费者组件持有并向内凹陷
`>` `<` `v`	依赖箭头	由消费者（需求方）指向提供者（实现方）

- 遗留语法与现代 UML 2.x 组件图的差别：2.x 用「提供接口（实心圆）+ 装配连接器」表达同一含义，而 1.x 直接把接口画在组件边界上。
- 五个组件与 9.3 类图中的服务一一对应：`界面层组件` ↔边界对象，`借阅业务组件` ↔ `LoanService`，`库存组件` ↔ `IS`，`读者认证组件` ↔ `AS`，`数据访问组件` ↔ `BookRepository`。
- 每个组件都通过 `数据访问接口` 依赖 `数据访问组件`，体现「统一持久化出入口」的架构约束。

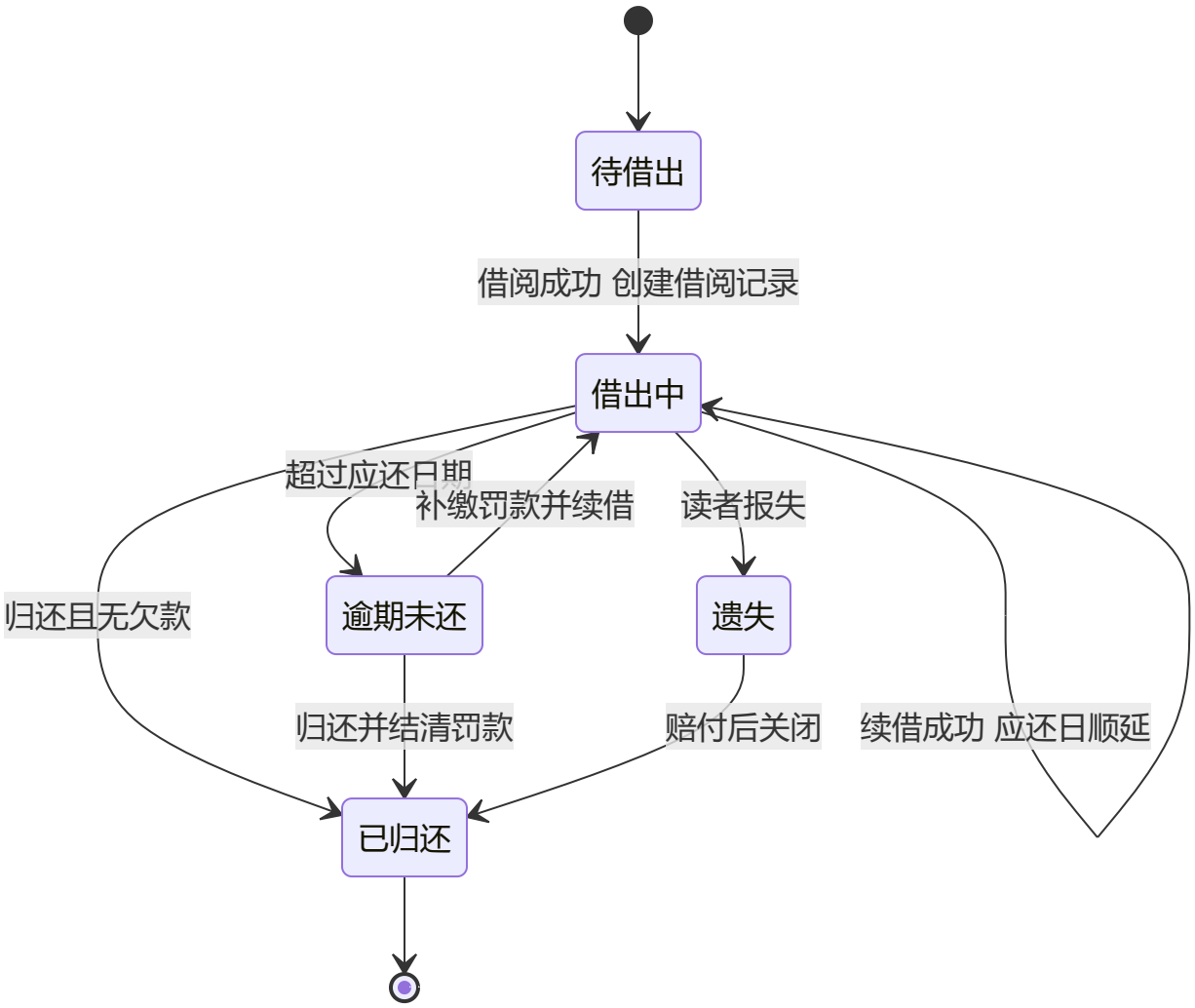
9.7 部署图（Deployment Diagram）



读图要点

- 图分四层：客户端层、应用层、数据层、外部系统；**边界即节点、连线即通信路径**，每条边标注通信协议。
- 应用层服务可多实例部署（`A1` `A2` `A3` 均为「实例」），与 9.6 组件图的组件划分一一对应。
- 数据层区分主库（读写）、只读副本（报表）、缓存（热点数据）、对象存储（非结构化封面），体现读写分离与存储分治。
- 外部系统只以单向调用接入，支付网关不持有业务状态，失败由应用层补偿。

9.8 状态图（State Diagram）



读图要点

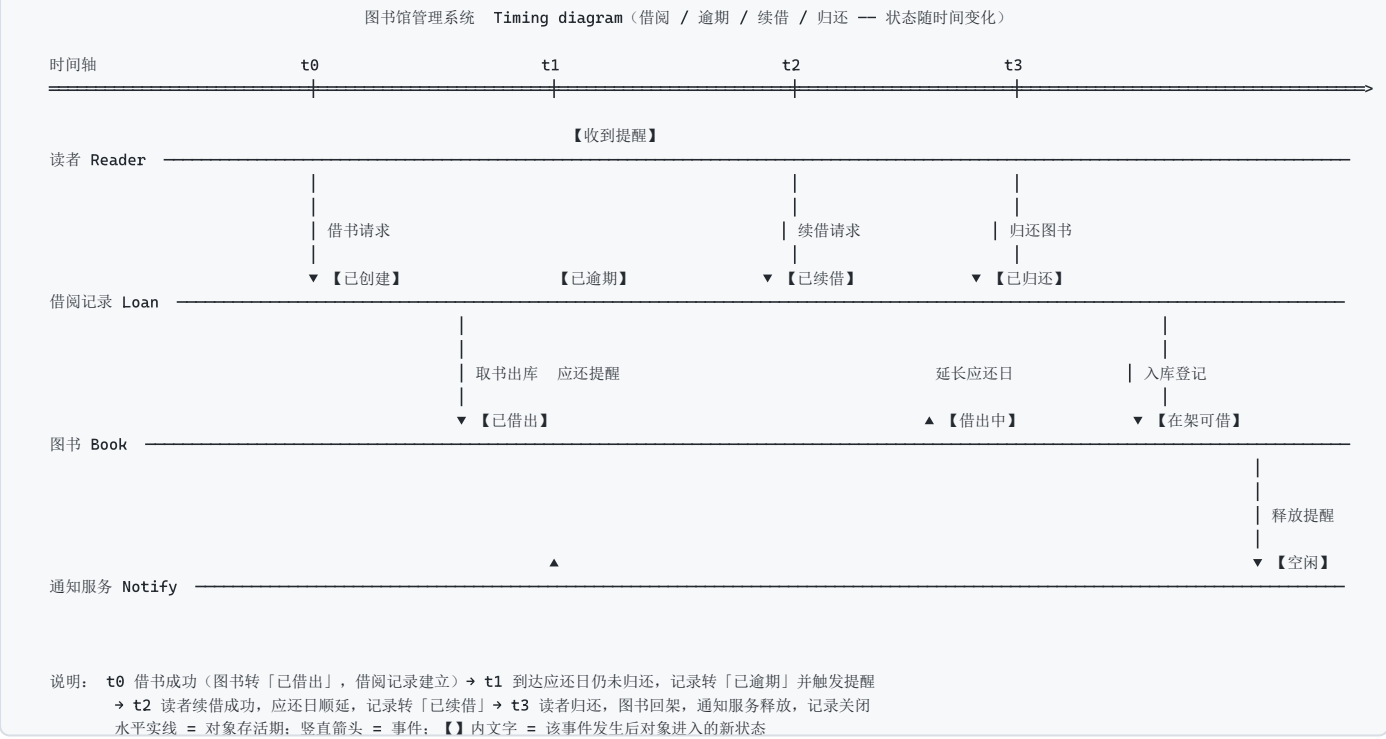
状态	含义	可迁出至
待借出	借阅记录已创建、副本已锁定	借出中
借出中	借阅生效，副本在读者手中	借出中（续借）、逾期未还、已归还、遗失
逾期未还	超过应还日期仍未归还，罚款待缴	借出中（缴清并续借）、已归还
遗失	读者报失，副本挂失	已归还（赔付后）
已归还	借阅终结，副本回架	终态

- **自环** `借出中 → 借出中` 表示续借事件，是本系统唯一不改变状态却改变数据（应还日）的迁移。
- **守卫条件**写在迁移标签上（借阅成功 / 续借成功 / 超过应还日期 / 归还且无欠款），与事件流中的判定节点一一呼应。

- 状态集与类图 `Loan.status` 的取值域一致，图书副本 `BookCopy.status` 另取「在架 / 借出 / 维修 / 下架」四值。

9.9 定时图（Timing Diagram）

定时图（Timing Diagram）强调**对象状态随时间的变化**，横轴为时间、纵轴为对象生命线，用事件箭头驱动状态迁移。



读图要点

- 水平实线表示对象的**存活期**；竖直箭头表示**事件**；`【】` 内文字为该事件后对象进入的新状态。
- 四个时间点构成完整生命周期：t_0 借书 → t_1 到期未还转逾期 → t_2 续借顺延 → t_3 归还闭环。
- 本图与 9.4 对象图互为印证：对象图是 t_1 时刻的静态快照，定时图给出到达该快照的**时间路径**。

9.10 图纸索引表

图号	图名	类型	主要读者	与本文档其他部分的关系
图 9-1	时序图	行为图	开发	消息序号对应第 5 章事件流步骤
图 9-2	用例图	行为图	需求 / 测试	用例编号延续第一部分 UC 编号体系
图 9-3	类图	结构图	开发	类名与 9.6 组件一一对应
图 9-4	对象图	结构图	开发 / 测试	为 9.9 的 t_1 时刻快照
图 9-5	活动图	行为图	开发 / 测试	判定节点对应事件流的异常流
图 9-6	组件图（遗留语法）	结构图	架构	组件名对应 9.7 部署单元
图 9-7	部署图	结构图	运维	部署单元对应 9.6 组件
图 9-8	状态图	行为图	开发 / 测试	状态取值与 9.3 `Loan.status` 一致
图 9-9	定时图	行为图	测试	状态迁移与 9.8 状态机一一对应

> 九张图覆盖 UML 2.x 的全部主要图类：**结构图**（类图、对象图、组件图、部署图）与**行为图**（用例图、活动图、状态图、时序图、定时图），可据此完成系统的完整可视化建模。

第一部分以腾讯 WeTest 微信小程序端「我的 → 账号与安全 → 修改用户名 / 修改密码」为分析对象；第二部分以图书馆管理系统为建模对象。